CSE305 Final Project Report
Parallel Hierarchical Agglomerative Clustering

Emeric Payer, Mateo Fatas, Matteo Sainton

June 7, 2026

Contents

1 Introduction

Hierarchical Agglomerative Clustering (HAC) is an unsupervised learning method that organizes a dataset into a tree of nested clusters. It starts with every point as its own cluster and repeatedly merges the two closest clusters (with respect to a set of rules) until a single cluster remains. The full merge history forms a *dendrogram*: a tree that records which clusters merged and at what distance. Unlike K-Means, HAC does not require choosing the number of clusters in advance: the dendrogram captures the structure at every level of granularity at once.

The drawback is cost. A naive implementation runs in $O(n^3)$ time, and even with careful bookkeeping the best general algorithms run in $O(n^2)$. That is fast enough for small inputs but becomes a bottleneck as the dataset grows, which makes HAC a natural target for parallelization.

This project implements and parallelizes HAC on multi-core CPUs using native `std::thread` primitives (introduced in C++11). We cover two linkage criteria, *single-link* and *average-link*. They are mathematically different enough that the best way to parallelize each one is also different, and comparing the two is the central theme of the report.

1.1 Task Partition and Component Ownership

We split the work so that each of us owns a complete vertical slice of the project (algorithm, validation, and measurements that go with it):

- **Emeric Payer:** responsible for the single-link pipeline: the optimized sequential baseline (`single-link-baseline.cpp`), the parallel graph-based implementation (`single-link-mst.cpp`), and single-link validation.
- **Mateo Fatas:** responsible for the average-link matrix path: the sequential baseline, its naive thread-parallel implementation (`hac_naive`), and the associated performance logs.
- **Matteo Sainton:** responsible for the space-partitioning average-link implementation (`hac_ppop`), the automated validation harness, and the Python benchmarking utilities.

1.2 Generative AI Usage Acknowledgement

In line with the course policy, we used generative AI tools only to help structure and copy-edit the prose of this LaTeX report, to tidy repository inventory summaries, and generate simple datasets for initial testing purposes.

All C++ algorithms, parsing and synchronization code, the experimental setup, and the analysis of the results were written by the team members themselves, although AI has also been used to debug certain sections and to help review that we had not overlooked any sub-cases.

2 Repository and Architecture Overview

The codebase is split into two independent algorithm directories joined by a shared build interface. All implementations follow the same data contract: each binary reads headerless CSV vectors and writes a dendrogram in a common format. This shared format is what lets us validate every implementation against the others.

2.1 Data Format and Validation

Each input line is one point, given as comma-separated numeric coordinates. Every implementation writes its dendrogram using the same four-field schema:

```
cl1, cl2, dist, new_size
```

where `cl1` and `cl2` are the two merged cluster identifiers, `dist` is the Euclidean distance at which they merged, and `new_size` is the number of points in the resulting cluster.

Cluster identifiers can legitimately differ between implementations because of floating-point ties or a different merge order, so the validation scripts (`validate_single_link.py` and `validate_average_link.py`) never compare files line by line. Instead they sort both outputs by the pair (`dist`, `new_size`) and compare the sorted sequences. Two runs are considered equivalent if they merge the same cluster sizes at the same distances, regardless of labelling.

The workspace is laid out as follows:

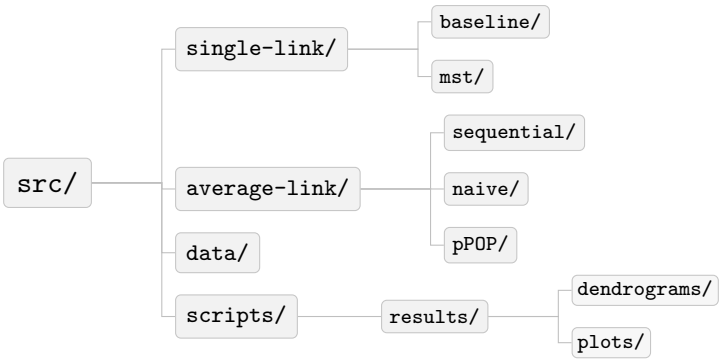


Figure 1: Repository layout

3 Single-Linkage Clustering

Under single linkage, the distance between two clusters A and B is the distance between their two closest points:

$$D(A, B) = \min\{d(x, y) : x \in A, y \in B\}.$$

As a consequence, when clusters i and j merge, any other cluster k whose nearest neighbour was i or j still has the merged cluster as its nearest neighbour, since the new distance is just the smaller of the two old ones. In other words, a merge never increases a nearest-neighbour distance, so we do not have to recompute nearest neighbours from scratch after every step. This is what makes an efficient $O(n^2)$ sequential algorithm possible, and it is also what lets us reformulate the whole problem as a graph problem for the parallel version.

3.1 Sequential Baseline: Nearest-Neighbour Matrix

The sequential baseline (`single-link-baseline.cpp`) implements the $O(n^2)$ algorithm of Olson [2] for the single-link metric. It maintains two data structures: a symmetric $n \times n$ distance matrix D and a nearest-neighbour array $N(i)$ storing, for each active cluster i, the index of its closest counterpart. Naïve HAC rescans all $O(n^2)$ pairs at every step, giving $O(n^3)$ overall; caching $N(i)$ reduces each global-minimum search to an $O(n)$ scan of that array instead.

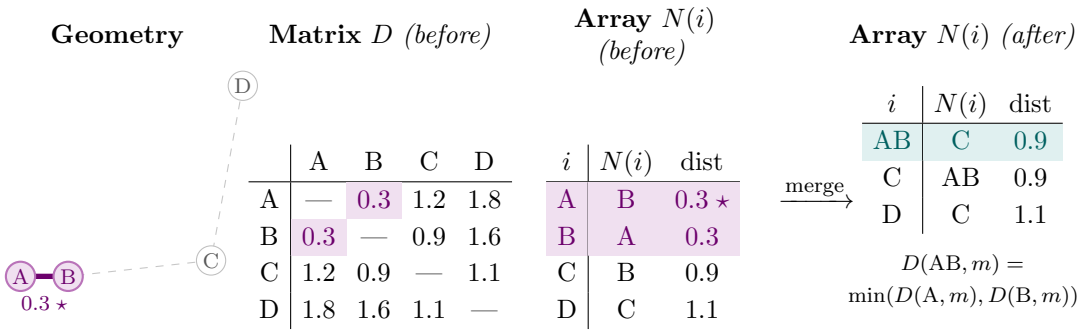
Each of the $n - 1$ merge steps proceeds as follows:

1. scan $N(i)$ in $O(n)$ to find the globally closest pair (i^*, j^*)
2. create a new cluster and compute its distances to all remaining clusters in a single $O(n)$ pass using the Lance–Williams update, which for single link reduces to

$$D(\text{new}, m) = \tfrac{1}{2}D(i^*, m) + \tfrac{1}{2}D(j^*, m) - \tfrac{1}{2}\,|D(i^*, m) - D(j^*, m)| = \min\big(D(i^*, m), D(j^*, m)\big)$$

3. update $N(i)$ for each surviving cluster: those whose cached nearest neighbour was neither i^* nor j^* need a single comparison against the new entry; those whose nearest neighbour was i^* or j^* take the merged cluster as their new nearest neighbour directly, since under single linkage its distance is just the smaller of the two old ones ($O(1)$ assignment)

Steps (1)–(3) are each $O(n)$, giving $O(n^2)$ overall.



	A	B	C	D
A	—	0.3	1.2	1.8
B	0.3	—	0.9	1.6
C	1.2	0.9	—	1.1
D	1.8	1.6	1.1	—

i	$N(i)$	dist
A	B	0.3 ⋆
B	A	0.3
C	B	0.9
D	C	1.1

i	$N(i)$	dist
AB	C	0.9
C	AB	0.9
D	C	1.1

Figure 2: Visualization of the single-link baseline algorithm on four clusters

This pipeline will serve as both the correctness reference and the speedup baseline.

3.2 Parallel Implementation: Borůvka-Style MST

The parallel module (`single-link-mst.cpp`) exploits the equivalence between single-link clustering and the Minimum Spanning Tree (MST): merging the two closest clusters is identical to adding the cheapest cross-component edge, so the full dendrogram is encoded in the MST [3]. Borůvka's algorithm builds that MST in $O(\log n)$ rounds, each eliminating at least half the components, and maps cleanly onto shared-memory parallelism because every component searches for its cheapest outgoing edge independently.

Initially every point is its own component. Each round proceeds as follows:

1. the $\binom{n}{2}$ point pairs are partitioned by row across `std::thread` workers; each worker scans its slice and, for every inter-component edge found, updates a per-thread candidate buffer (using lock-free `find_root_read_only` to query component membership)
2. once all workers join, the main thread reduces the per-thread buffers into a single cheapest outgoing edge per component
3. the chosen edges are committed to a union–find structure (union by rank, path-compressing `find_root`); no synchronisation is needed as this phase is sequential

Once the MST is complete, its $n - 1$ edges are sorted by weight and replayed through a second union–find pass to emit the dendrogram in the shared output format.

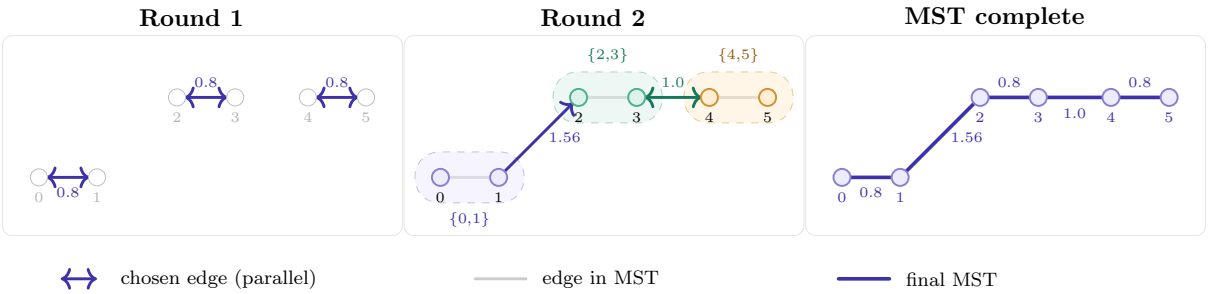


Figure 3: Two Borůvka rounds: six points pair up, then chain into one MST.

4 Average-Linkage Clustering

Under average linkage, the distance between clusters A and B is the mean distance over all cross-pairs of points:

$$D(A, B) = \frac{1}{|A|\,|B|} \sum_{x \in A} \sum_{y \in B} d(x, y). \tag{1}$$

Because this value depends on the sizes of both clusters, the nearest-neighbour shortcut from single linkage does not hold: the distance to the merged cluster is a size-weighted average of the two parent distances, so it can exceed a cluster's previous nearest-neighbour distance, and any cluster whose nearest neighbour was one of the merged clusters must therefore be re-examined after each step.

4.1 Sequential Baseline

The sequential baseline keeps the full $O(n^2)$ dissimilarity matrix D between active clusters. At each of the $n - 1$ steps it scans the matrix for the closest pair (i, j), merges them into a new cluster k, marks i and j inactive, and updates the distances from k to every remaining cluster m with the average-link case of the Lance–Williams recurrence,

$$D(k, m) = \frac{|i|\,D(i, m) + |j|\,D(j, m)}{|i| + |j|}, \tag{2}$$

where $|i|$ denotes the number of points in cluster i. Finding the closest pair is $O(n^2)$ per step in the simplest version, giving $O(n^3)$ overall; this is the correctness reference and the speedup baseline for both parallel average-link implementations. A faster $O(n^2)$ sequential average-link algorithm exists (via nearest-neighbour chains); our baseline is the simpler $O(n^3)$ version, so the reported speedups are relative to it rather than to the best-known sequential algorithm. Because it stores the matrix explicitly, its memory cost is $O(n^2)$, which is what limits the reachable problem size on a single machine.

4.2 Naive Thread-Parallel Implementation

The naive parallel version (`hac_naive`) keeps the same stored-matrix algorithm and parallelizes both expensive steps of each iteration – the search for the globally closest pair and the subsequent row update – while leaving the merge bookkeeping sequential. Each iteration proceeds in three phases:

1. **Parallel find-min.** Matrix rows are handed out dynamically through a shared `std::atomic<int>` counter: each `std::thread` worker repeatedly claims the next available row with `fetch_add`, which balances load automatically even as active rows become sparse with each merge. Every worker keeps a private local-minimum slot.
2. **Sequential reduction.** Once the workers join, the main thread reduces these slots into the global minimum and performs the merge bookkeeping.
3. **Parallel row update.** The Lance–Williams update (2) for the new cluster row is itself parallelized: the affected target clusters are split into contiguous chunks and processed by the same worker pattern, with disjoint writes that need no synchronization.

The sequential fraction is therefore the per-step reduction and bookkeeping, plus the `std::thread` spawn/join cost, which here is paid twice per merge – once for the find-min workers and once for the update workers – and is visible in the small-n measurements.

4.3 Space Partitioning (pPOP)

The space-partitioning implementation (`hac_ppop`) follows the nested pPOP algorithm of Dash et al. [1], adapted from their centroid measure to average linkage. The idea is to avoid comparing every cluster against every other by exploiting geometry: the 2-D bounding box of the data is divided into a $c \times c$ grid of cells, and a cluster only ever needs to be compared against clusters in the same cell.

Overlapping cells. To make this exact, adjacent cells are made to overlap. Each cell boundary is extended by $\delta/2$ on every side, so neighbouring cells share a band of width δ. This guarantees the key invariant: *any two clusters closer than δ share at least one cell*, so a merge that should happen at distance below δ is never missed. Correctness therefore depends only on this overlap, not on the number of cells.

Per-cell priority queues. Within a cell, each cluster keeps a min-heap (`std::priority_queue`) of its distances to the other clusters in that cell. Finding the closest pair becomes a scan of heap tops over the cells rather than over the whole matrix. Stale entries (referring to clusters that have since merged) are removed lazily: when a heap top points to an inactive cluster it is popped and the next candidate examined. To keep the per-cell heaps race-free under parallel access, a cluster's heap is only ever read or written by the owner of its *primary* cell (the first cell in its membership list), which guarantees that each heap is touched by exactly one thread.

Nested schedule. The algorithm runs in nested iterations. It begins with $\delta = 0$ and the initial grid; the first iteration therefore merges nothing and simply records the smallest available inter-cluster distance, which seeds δ for the next iteration. Each subsequent iteration repeatedly merges the closest pair while the smallest available distance stays below δ, and ends once that distance exceeds δ. Following the spirit of the "90–10" rule of the paper, between iterations we grow δ to 1.1 times the distance that ended the previous iteration ($\delta \leftarrow 1.1\, d_{\text{exceed}}$) and coarsen the grid by reducing the number of cells per dimension by roughly 10% ($c \leftarrow \lfloor 0.9\, c \rfloor$, floored at a small minimum tied to the thread and cluster counts). The grid and heaps are rebuilt at the start of each iteration. The final iteration, with the coarsest grid, plays the role of the paper's second phase and merges whatever remains.

Deviation from the paper: global distance update. The paper uses the centroid measure, for which a merged cluster has a single representative point; this lets it update only the clusters in the container cell after a merge. Average linkage has no representative point: by (2) the new distances depend on the old pairwise distances to *both* parents, and these must stay globally correct because a later iteration may compare the merged cluster against clusters that were in other cells. We therefore keep the Lance–Williams update global (over all active clusters) rather than container-cell-local. This is the one substantive deviation from the paper, and it is dictated by the choice of linkage, not by the parallelization.

Parallelization. Four parts of each iteration run on a persistent pool of `std::thread` workers built once at the start (a condition-variable pool, so threads sleep between tasks):

1. the per-iteration construction of the cell heaps
2. the per-cell closest-pair search
3. the global Lance–Williams update, parallelized over the list of target clusters
4. the update of affected clusters' heaps after a merge

The overall-minimum reduction and the merge bookkeeping remain sequential.

5 Experimental Evaluation

The experiments aim to answer three questions: how each parallel implementation scales as the thread count grows, at what dataset size the parallel versions overtake their sequential baselines, and how the two linkage strategies compare on the same hardware.

5.1 Benchmarking Environment

All benchmarks and thread-scaling runs are executed on a single shared-memory machine.

- **Machine:** Apple MacBook Air (Apple silicon)
- **Cores:** 8 cores (mixed performance and efficiency), used as up to 8 software threads
- **Compiler:** GNU `g++`, `-O2`, C++20 standard

Each configuration is run three times and the minimum wall-clock time is reported, which suppresses transient scheduling noise. Each timing measures the clustering work itself: it starts after parsing finishes and stops before the output is written to disk.

5.2 Scaling Analysis

Inputs are synthetic clustered datasets from `generate_data.py` (ten Gaussian blobs) at $n \in \{500, 1000, 2000, 5000\}$. The table below collects representative timings and speedups; the full thread-scaling curves are in Figures 4b, 5b and 5c.

Dataset	Algorithm	Threads	Runtime	Speedup	Pred. (Amdahl)
Synthetic ($n = 5000$)	Sing.-link base	1 (seq)	319.7 ms	1.00×	—
	Parallel MST	2	194.9 ms	1.64×	—
	Parallel MST	4	117.3 ms	2.73×	—
	Parallel MST	8	68.0 ms	4.70×	—
Synthetic ($n = 5000$)	Avg-link base	1 (seq)	21774.1 ms	1.00×	—
	Avg-link naive	2	13971.5 ms	1.56×	1.82×
	Avg-link naive	4	7962.4 ms	2.73×	3.10×
	Avg-link naive	8	6401.3 ms	3.40×	3.40×
	Avg-link pPOP	1	2856.2 ms	7.62×	—
	Avg-link pPOP	2	2355.4 ms	9.24×	—
	Avg-link pPOP	4	2117.4 ms	10.28×	—
	Avg-link pPOP	8	1248.2 ms	17.44×	—

Table 1: Execution time (ms) and speedup across thread counts

Single-link. The MST scales to about 4.7× at $t = 8$ for $n = 5000$. The gap from ideal linear speedup comes from the serial reduction-and-union step that each round interleaves with its parallel edge search: at these sizes, the parallel portion is not large enough to dominate it, so by Amdahl's law the serial fraction bounds the achievable speedup. Since the problem is inexpensive in absolute terms (sub-second even sequentially), this sub-linear speedup is acceptable.

Average-link. The naive matrix version scales modestly (about 3.4× at $t = 8$, $n = 5000$), because its parallel fraction – the find-min scan and row update – is flanked each step by serial work. pPOP instead starts from a much lower single-thread time, since the spatial partitioning does strictly less work; its per-thread scaling is actually slightly weaker than the naive version's, so its advantage over the baseline is overwhelmingly algorithmic rather than parallel.

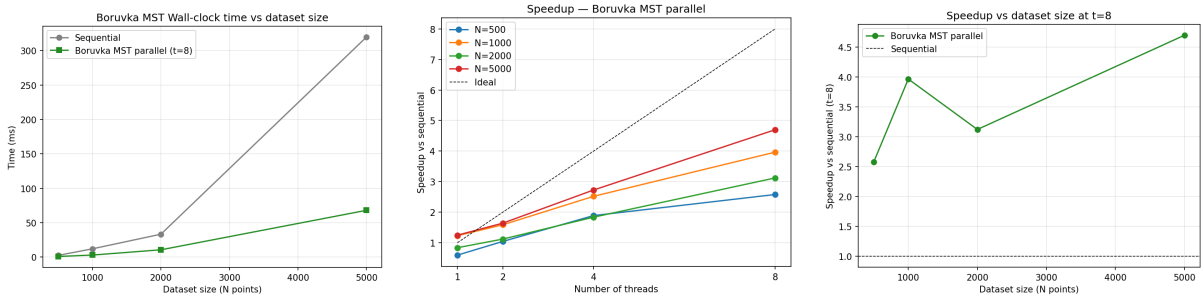
5.3 Experimental Performance Measurements

5.3.1 Single-link

Figure 4a shows wall-clock time against dataset size and Figure 4b the speedup against the sequential baseline as the thread count grows. The single-link problem is intrinsically cheap: even the sequential baseline clusters $n = 5000$ points in roughly 320 ms, because the $O(n^2)$ algorithm performs only simple, cache-friendly array scans. The parallel MST nevertheless delivers a clear and consistent benefit at every size large enough to amortize thread management, reaching about 4.7× on eight threads at $n = 5000$ (319.7 ms down to 68.0 ms).

Three features of the data are worth highlighting:

1. at $n = 500$, the single-thread MST is slower than the sequential baseline (0.59×): the Borůvka formulation does more bookkeeping per merge (Union–Find, per-round edge buffers) than a flat matrix scan, and on a single thread there is no parallel gain to offset this (the overhead is only repaid once enough threads exploit the parallel edge search)
2. the speedup sits well below the ideal line, as expected: only the per-round edge search is parallel, while the reduction over per-thread buffers and the Union–Find merge are short sequential phases, so by Amdahl's law the serial fraction caps the achievable speedup
3. the speedup-vs-size curve at $t = 8$ (Figure 4c) is not monotone – the $n = 2000$ point dips below $n = 1000$. The number of Borůvka rounds depends on the structure of the data (how quickly components coalesce), not only on n; since each round carries a fixed serial reduction-and-union cost, the serial fraction – and hence the speedup – varies slightly between inputs of similar size.



(a) Time vs. dataset size (b) Speedup vs. thread count (c) Speedup vs. dataset size

Figure 4: Single-link clustering performance

5.3.2 Average-Link

Average-link naive. At $n = 500$ the naive matrix version stays below 1× for all thread counts and even degrades as threads are added: the per-step dispatch and reduction cost more than the find-min scan saves on a tiny matrix. As n grows the find-min scan dominates and parallelization pays off, reaching 3.4× at $t = 8$, $n = 5000$. The sub-linear ceiling follows from Amdahl's law: the parallel phases (find-min scan and Lance–Williams update) are flanked each step by serial work – the $O(n)$ target-list rebuild, the minimum reduction, and the spawn/join cost paid twice per merge. Fitting Amdahl's law to the $t = 8$ point gives a serial fraction $f \approx 0.10$; the predictions (Table ??) exceed the measured speedups at $t = 2$ and $t = 4$, consistent with the per-merge spawn/join overhead growing with thread count rather than staying fixed as the model assumes.

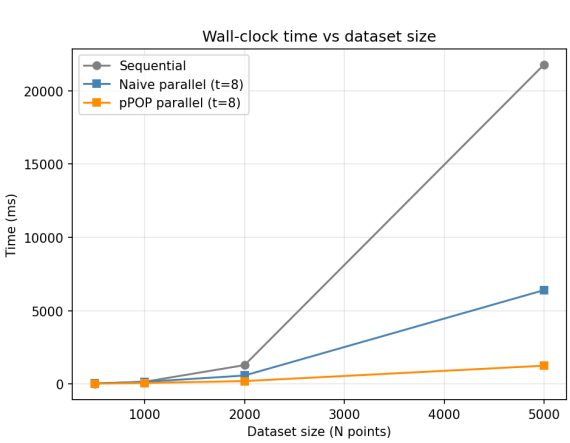
Average-link pPOP. pPOP behaves qualitatively differently. Two effects compound:

1. Algorithmic gain: it is far faster than the sequential baseline (7.6× at $n = 5000$), as the spatial partitioning means each cluster is only compared against the handful of clusters in its cell, so there are fewer cluster comparisons per thread
2. Parallel gain: the threads divide the remaining work, taking the speedup to 17.4× at $t = 8$

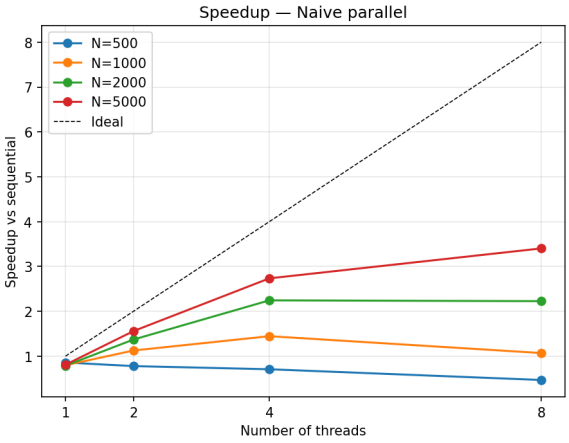
The two factors compose: the speedup over the sequential baseline equals pPOP's single-thread algorithmic gain ($\approx 7.6\times$) times its own parallel speedup across threads ($\approx 2.3\times$ at $t = 8$), which is why the speedup-vs-sequential figure exceeds the thread count – it conflates an algorithmic improvement with a parallel one.

The same figure also shows the characteristic small-n behaviour: at $n = 500$ pPOP gives no thread benefit (and dips to $0.81\times$ at $t = 8$), because at small n there are too few clusters to keep more than one cell populated once the run is underway, so the grid reduces to a single cell and the work can no longer be spread; the speedup grows steadily with n as this single-cell endgame becomes a smaller fraction of the run.

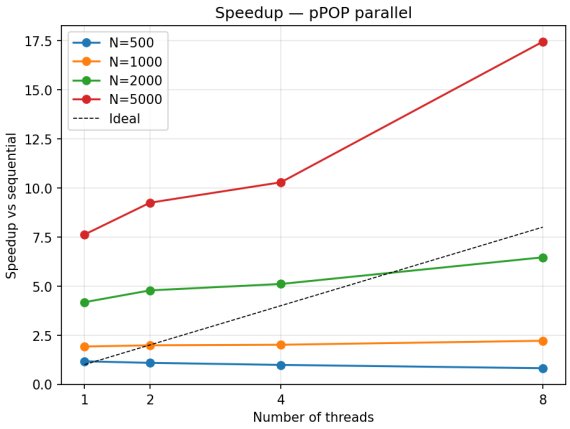
Cross-strategy comparison. Figures 5a and 5d put the two average-link strategies side by side. At $t = 8$, pPOP outperforms the naive version at every dataset size, and the gap widens rapidly with n, exactly as the partitioning argument predicts. The absolute-time plot makes the practical point most clearly: at $n = 5000$ the sequential baseline takes about 21.8 s, the naive parallel version about 6.4 s, and pPOP about 1.2 s, the latter two at $t = 8$.



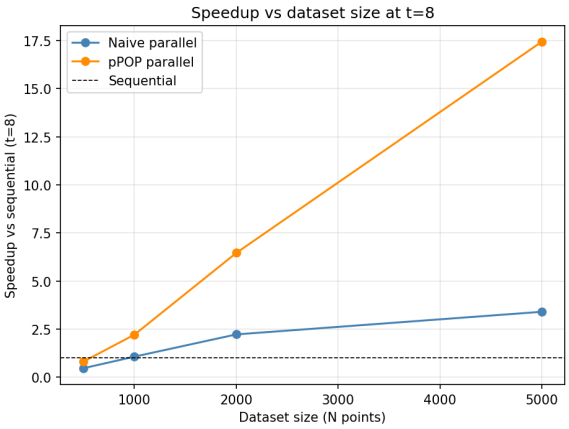
(a) Wall-clock time vs. dataset size



(b) Naive: speedup vs. thread count



(c) pPOP: speedup vs. thread count



(d) Speedup at $t = 8$ as a function of dataset size

Figure 5: Absolute time and thread scaling for the naive and pPOP implementations

5.4 Cluster Quality and Linkage Limitations

The thread-equivalence check confirms that each parallel implementation reproduces its sequential baseline, but says nothing about whether the clustering is correct. To check this we cut each dendrogram at the dataset's known cluster count and plot the recovered labels against the raw points. On well-separated Gaussian blobs every implementation agrees (Figure 6): the five panels are identical up to a few boundary points, confirming both cross-implementation equivalence and qualitative correctness when the geometry is easy.

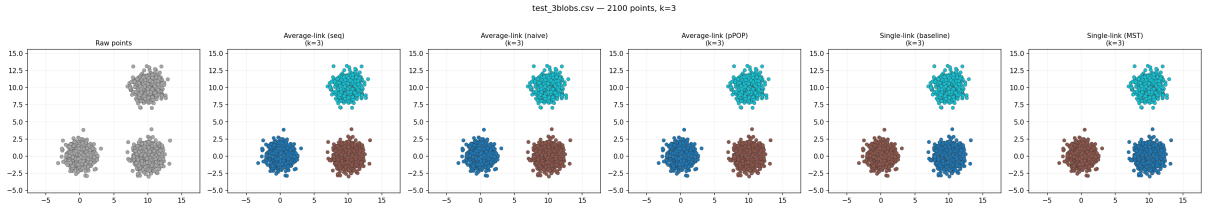


Figure 6: Recovered clusters at $k = 3$ on a synthetic 3 blobs dataset ($n = 2100$)

The differences predicted by our algorithms appear only when the data violates the assumption each linkage relies on. The two cases are opposite, and each linkage fails on exactly the geometry the other handles.

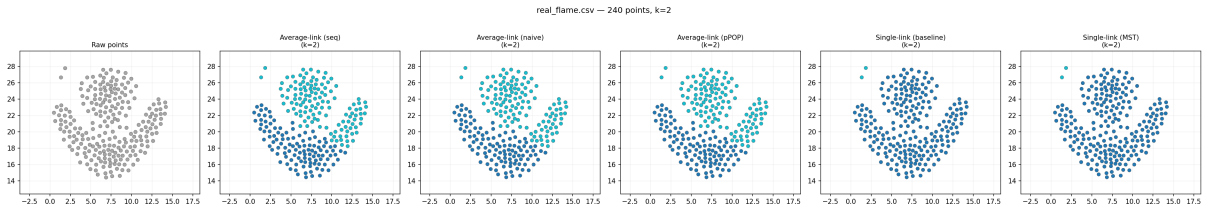


Figure 7: `flame` ($k = 2$): two interlocking non-convex bands

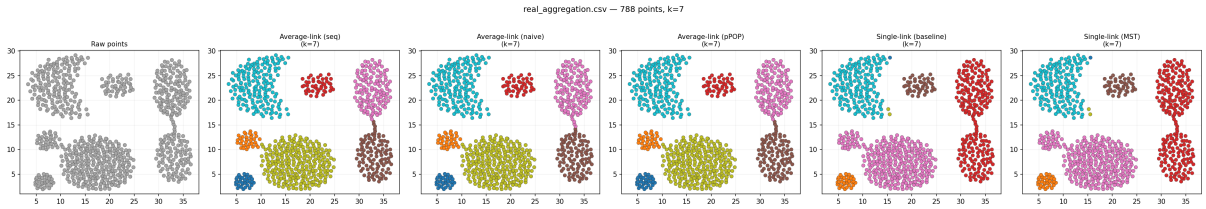


Figure 8: `aggregation` ($k = 7$): two dense groups joined by a thin bridge

These two failures are the same property seen from both sides.

1. Single-link merges on the *minimum* cross-pair distance, so a sparse chain of points fuses clusters that should stay separate (Figure 8) but a non-convex band is followed faithfully (Figure 7).

2. Average-link merges on a size-weighted *mean* (Equation 2), so compact groups are recovered cleanly but elongated or bridged structure is cut. On the dense-blob datasets (R15, D31) the same effect is visible more mildly: average-link separates the tight groups, while single-link fuses neighbouring blobs and over-splits isolated ones.

Neither linkage dominates; the right choice is dictated by the data geometry – the clustering-quality counterpart to the report's central point that the right *parallelization* follows from the structure of the linkage.

6 Conclusion

This project set out to parallelize hierarchical agglomerative clustering under two linkage criteria, and its central lesson is that the right parallelization follows from the structure of the linkage rather than from a generic threading recipe.

Single linkage carries a special property – a merge never increases a nearest-neighbour distance – which makes the dendrogram an MST and a parallel Borůvka algorithm the natural fit. The result is a solid, if sub-linear, speedup (about 4.7× on eight threads), bounded by the serial reduction-and-union phase between parallel rounds, on a problem that is already inexpensive in absolute terms. Average linkage offers no such shortcut: distances must be recomputed after every merge, so the naive matrix version can parallelize only its find-min scan and is capped near 3.4×.

The space-partitioning pPOP method is the clear winner for average linkage, but for a reason worth stating precisely: most of its advantage is algorithmic rather than parallel. Restricting comparisons to overlapping spatial cells makes it several times faster than the baseline on a single thread, and the threads then multiply that gain – which is why its speedup over the sequential baseline (17.4× at $n = 5000$) exceeds the thread count. Adapting it from the paper's centroid measure to average linkage cost us the container-cell-local distance update, which we had to make global, but the partitioning advantage on the dominant find-min and heap-construction steps survived that change.

The one caveat is dimensionality: the grid partitioning is two-dimensional, so the method as implemented does not extend directly to high-dimensional data, where the cell structure would have to be replaced by a spatial index.

References

[1] M. Dash, S. Petrutiu, and P. Scheuermann. pPOP: Fast yet accurate parallel hierarchical clustering using partitioning. *Data & Knowledge Engineering*, 61(3):563–578, 2007.

[2] Clark F. Olson. Parallel algorithms for hierarchical clustering. *Parallel Computing*, 21(8):1313–1325, 1995.

[3] Sanguthevar Rajasekaran. Efficient parallel hierarchical clustering algorithms. *IEEE Transactions on Parallel and Distributed Systems*, 16(6):497–502, 2005.